

Automated Maze Solver: Progress Report & Adjusted Strategy

Dipra

September 28, 2026

1 Introduction

With only two days remaining until the project submission deadline, this report outlines the significant progress made on the Automated Maze Solver, details the recurring hardware and software challenges encountered, and presents a revised, focused strategy to ensure a successful and complete demonstration.

2 Progress Achieved

Considerable milestones have been reached in developing the dual-microcontroller architecture of the maze solver:

- **Dual-MCU Architecture:** Successfully established a robust master-slave configuration. The ESP32 acts as the high-level brain handling sensors and navigation algorithms, while the ATmega32 acts as the motor controller, receiving commands via UART.
- **Sensor Integration:** Successfully interfaced three VL53L0X Time-of-Flight (ToF) sensors over a shared I2C bus using XSHUT pins for dynamic address assignment.
- **Gyroscope Tracking:** Integrated the MPU6050 for heading tracking. Implemented a Proportional-Derivative (PD) control loop allowing the car to maintain a straight trajectory and dynamically correct for motor imbalances.
- **Telemetry & Debugging:** Built a comprehensive Bluetooth telemetry system that mirrors internal ESP32 logs to a mobile device, allowing for real-time debugging without physical cable tethers.
- **Fault Tolerance:** Implemented a software fallback mechanism (Timed Turns) and I2C bus recovery sequence (9-clock cycle SCL pulse) to recover from mid-operation hardware freezes.

3 Major Recurring Challenges

The project has faced persistent challenges, predominantly stemming from hardware and electrical limitations:

- **Electrical Noise and I2C Bus Instability:** The most significant hurdle has been electrical noise generated by the brushed DC motors. During sudden power draws (especially during pivoting turns), voltage spikes/drops cause the sensitive I2C bus to crash, leading to "Error 263" and freezing the Gyroscope (MPU6050) and ToF sensors.
- **Motor Asymmetry:** The right and left motors exhibit different physical torque characteristics, causing the car to naturally drift. While the PD controller attempts to compensate, extreme corrections have sometimes led to runaway oscillations.
- **Sensor Lockups During Turns:** Because the I2C bus freezes during aggressive turns, the closed-loop gyro turning algorithm loses its feedback. We successfully implemented a software fallback (open-loop timed turns), but physical calibration remains sensitive to battery voltage.

4 Latest Troubleshooting & Hardware Redesign

In the final stages of development, several critical hardware bottlenecks were identified and successfully resolved:

- **ESP32 Brownouts & Boot Loops:** We observed that the car would hang upon booting on battery power, taking upwards of a minute to establish a Bluetooth connection. Analysis of the logs revealed that the ESP32 was suffering from brownouts (voltage drops causing sudden reboots) at the exact moment the 2.4GHz Bluetooth radio initialized, due to the sudden current spike.
- **I2C Sensor Freezes:** When the ESP32 did manage to boot, the Front Time-of-Flight sensor would consistently fail to initialize. The I2C bus was severely corrupted by electrical noise, causing the sensor library's initialization function to block for 40+ seconds waiting for a response that would never arrive.
- **Dual-Battery Power Architecture (The Fix):** To eliminate these power starvation and noise issues, the entire power architecture was redesigned into a dual-battery setup. A primary logic battery now connects to a 5V Buck Converter, cleanly powering the breadboard, ATmega32, and ESP32. A separate 7.4V battery pack directly powers the motor driver (L298N). The heavy motor current is now fully isolated from the sensitive logic circuit.
- **Noise Filtering Capacitors:** A 1000 μ F decoupling capacitor was added across the motor driver, and 100nF ceramic capacitors were added across the ESP32 and ATmega32 power pins to filter out the remaining high-frequency brush noise from the motors.
- **Non-Blocking Sensor Fallbacks:** The ESP32 firmware was rewritten to safely handle sensor failures. If a sensor fails to boot within a 3-second timeout, the system flags it as failed, bypasses its reading in the main loop (assuming maximum distance), and continues operating without freezing the entire MCU.

5 Revised Strategy and Reduced Scope

Given the hard two-day deadline, the project scope has been tightened to guarantee a functional demonstration. The strategy has shifted from a complex Gyro-assisted PID navigation system to a robust, logic-driven approach.

5.1 Demonstration Goals

1. **Memory Saving (ATmega32):** Highlight the dual-MCU architecture as a memory-saving strategy. Show how offloading the heavy navigation algorithms and Bluetooth stack to the ESP32 allows the ATmega32 to run a lightweight motor driver loop.
2. **Robust Sensor Navigation (Phase 5):** We have implemented a new, streamlined firmware that relies purely on the Time-of-Flight sensors, entirely bypassing the Gyroscope to avoid any lingering I2C vulnerabilities.
3. **Intersection Branching Logic:** The car uses a strict “Right-Hand Preference” algorithm. Upon detecting a frontal obstacle within 200mm, the car halts, analyzes the left and right sensors, and prioritizes turning into the rightmost open path. If both are blocked, it successfully executes a 180-degree U-turn.

5.2 Execution Plan

- **Open-Loop Calibration:** Rather than using Gyro feedback, turns are now purely time-based. The exact `delay()` durations for 90-degree and 180-degree maneuvers will be physically calibrated on the demonstration surface.
- **Simple Maze Layout:** Use a short track with clear, straight walls and defined decision points (T-junctions and Dead Ends) to demonstrate the branching logic flawlessly.

6 Conclusion

The underlying logic and architecture of the maze solver are fundamentally sound and sophisticated. By isolating the hardware power issues and shifting the software focus toward a robust, purely sensor-based intersection logic, the project can be successfully demonstrated within the remaining timeframe.

7 Hardware Connections Map

The final wiring architecture is strictly separated into logic and high-current motor pathways to prevent electrical interference.

7.1 Logic Power (Breadboard & MCUs)

- Battery Pack 1 (Logic) → Buck Converter IN
- Buck Converter (5V) OUT+ → Breadboard Power 1 & 2 (5V Rail)

- Buck Converter (5V) OUT- → Breadboard Gnd 1 & 2 (Common Gnd)

7.2 Motor Power (High Current)

- Battery Pack 2 (7.4V) positive → Rocker Switch (Terminal 1)
- Rocker Switch (Terminal 2) → Motor driver 12V
- Battery Pack 2 (7.4V) negative → Motor driver GND
- **CRITICAL:** Motor drive GND → Breadboard Gnd 2 (Common ground required for PWM logic!)
- *Note: Motor Driver 5V output is NO LONGER connected to the breadboard.*

7.3 Capacitors (Noise Filtering)

- 1000 μ F Capacitor → Across Motor driver 12V and GND
- 100nF Capacitor → Across ESP32 VIN and GND
- 100nF Capacitor → Across ATmega32 VCC and GND

7.4 Motor Driver Connections

- Out1 → Left Motor top
- Out2 → Left Motor bottom
- Out3 → Right Motor top
- Out4 → Right Motor bottom
- IN1 → ATmega PB0 (1)
- IN2 → ATmega PB1 (2)
- IN3 → ATmega PB2 (3)
- IN4 → ATmega PB4 (5)
- ENA → ATmega PD5 (19)
- ENB → ATmega PD4 (18)

7.5 ATmega32 Connections

- VCC (Pin 10) → Breadboard Power 1
- GND (Pin 11) → Breadboard Gnd 1
- AVCC (Pin 30) → Breadboard Power 2
- GND (Pin 31) → Breadboard Gnd 2

7.6 ESP32 Connections

- VIN → Breadboard Power 2
- ESP Gnd 1 → Breadboard Gnd 1

7.7 Logic Level Shifter (UART Bridge & 3.3V Sensor Power)

- HV → Power 1 (ATmega 5V)
- LV → ESP32 3V3 Pin (Creates the 3.3V Breadboard rail that powers all sensors)
- GND (both) → Breadboard Gnd 1 / Gnd 2
- LV1 → ESP32 TX2 (Pin 24)
- HV1 → ATmega RXD (PD0 / Pin 14)

7.8 I2C Sensors (3x VL53L0X ToF)

- **Power:** VIN/VCC → 3.3V Breadboard rail; GND → Common Ground.
- **I2C Bus:** SDA → ESP32 Pin 21; SCL → ESP32 Pin 22.
- **XSHUT Pins:** Front → ESP32 Pin 19; Left → ESP32 Pin 18; Right → ESP32 Pin 4.

8 Original Project Proposal Plan

This section outlines the initial goals, updates, and testing criteria presented during the project proposal phase.

8.1 Expected Outcome

A fully autonomous robot capable of navigating a 16x16 maze grid, mapping dead ends, and calculating the fastest route.

8.2 Implementation Plan & Updates

- **Weeks 1-2:** Complete chassis assembly, establish PC simulation of the Flood Fill algorithm, and finalize the ATmega32 PID controller for straight-line driving.
- **Weeks 3-5:** Integrate the ToF sensors with the ESP32, merge the mapping logic with the movement logic, and finalize edge-case tuning for wall-drift correction.

8.3 System Testing Methodology

The system was designed to be tested in 4 phases:

- **Phase 1 (Locomotion & PID):** Test perfectly straight 1m driving and exact 90° turns using encoder counts.
- **Phase 2 (Sensor & I2C Calibration):** Verify XSHUT addressing to prevent I2C conflicts and calibrate ToF distances.
- **Phase 3 (Algorithm Simulation):** Test Flood Fill algorithm on a PC simulation before flashing the ESP32.
- **Phase 4 (Full Integration):** Test hardware/software synchronization in scaled-down physical mazes.

8.4 Measurable Criteria for Success

- **PID Synchronization Error:** Evaluates steady-state error between left and right encoder counts to ensure straight-line stability.
- **Algorithmic Path Efficiency:** Evaluates optimal path calculation after mapping (Optimal cells vs. actual cells traversed).

8.5 Separation of Features

- **Essential Features (MVP):** Maintain accurate digital coordinates/heading, autonomously map the maze and find the center, and execute stable PID motor control.
- **Optional Features (Stretch Goals):** Return to start and execute a high-speed run through the optimal path, and broadcast live Wi-Fi telemetry to a dashboard.